

Customer Sales ETL Pipeline Documentation

Project Overview

Customer Sales ETL Pipeline with Data Quality Validation and Business Analysis
Objective

The objective of this project is to build a complete ETL (Extract, Transform, Load) pipeline that processes customer sales data from raw CSV files into clean, validated datasets and stores them in a MySQL database.

The project also generates:

- Clean datasets
- Rejected datasets
- Sales Transaction dataset
- Data Quality Report
- Business Analysis using SQL queries

This project simulates how real companies clean, validate, and analyze customer sales data before using it for reporting and decision-making.

2. Technology Used

Programming Language

- Python 3.x

Database

- MySQL

Libraries Used

Library	Purpose
pandas	Data loading and transformation
numpy	Missing value handling
mysql-connector-python	Connect Python with MySQL
logging	Pipeline logging
os	File and folder management
random	Mock data generation
faker	Generate fake customer data
sqlparse	Execute multiple SQL queries from SQL file

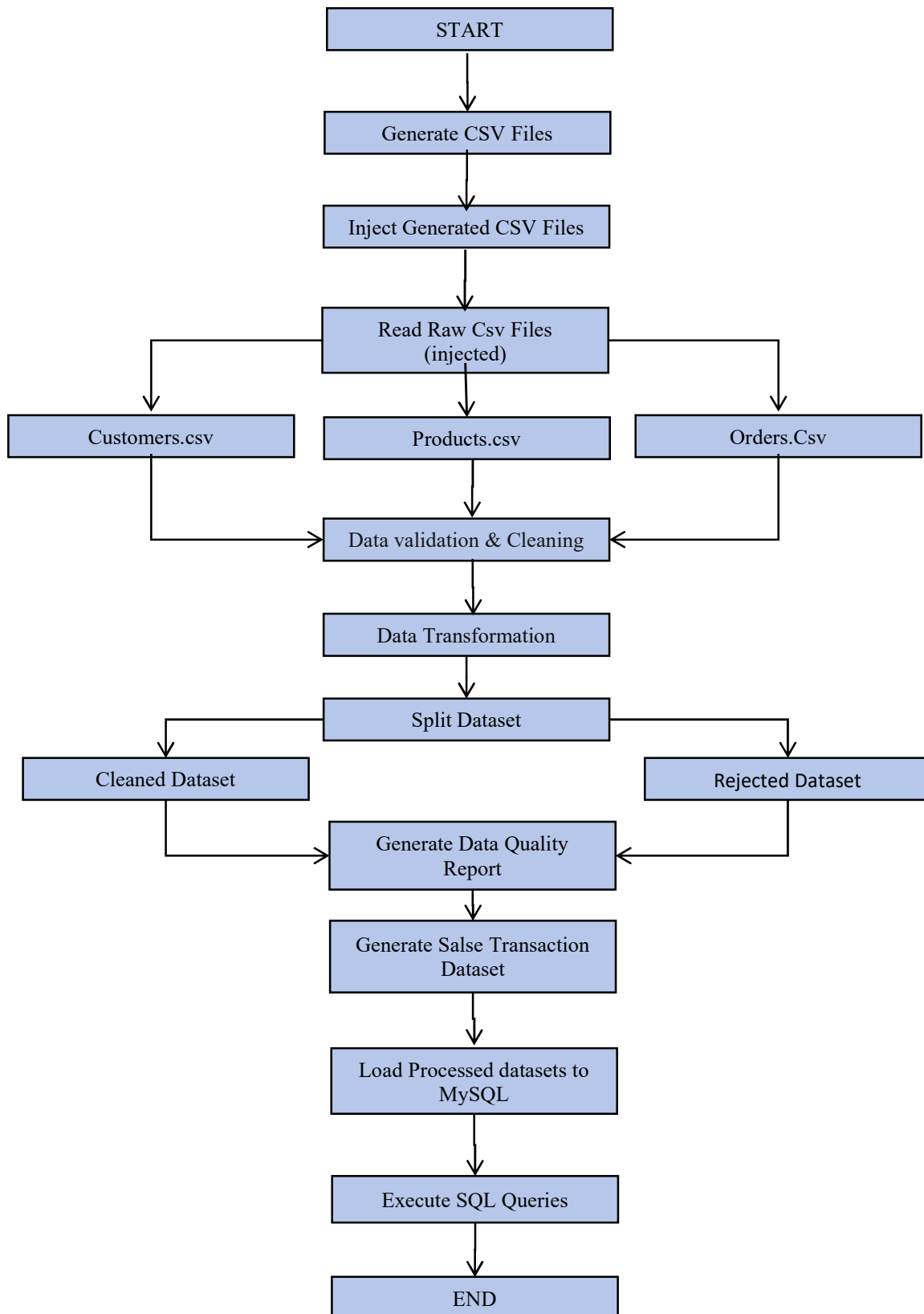
Input Format

- CSV Files

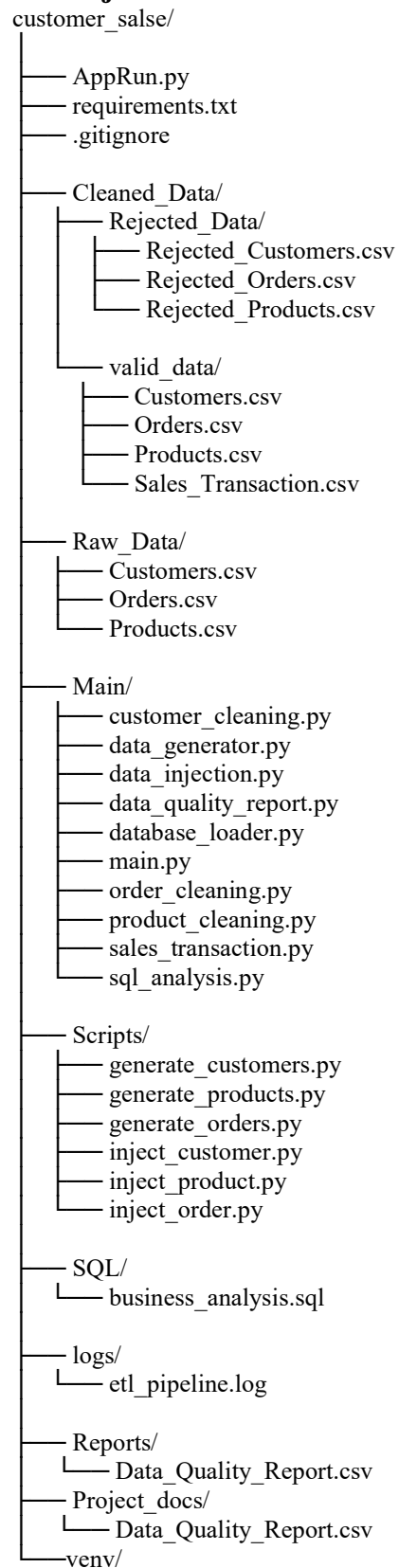
Output Formats

- CSV
- MySQL Tables
- Log File
- SQL Analysis Output

Workflow



3. Project Folder Structure



Input Data Sources and File Details

Customers Dataset

Attribute	Data Type	Description
Customer_ID	String	Unique identifier for each customer (Primary Key).
Customer_Name	String	Name of the customer.
Email	String	Customer's email address.
Phone	String	Customer's contact number.
Gender	String	Gender of the customer.
City	String	City where the customer resides.
State	String	State where the customer resides.
Registration_Date	Date	Date on which the customer registered.

Products Dataset

Attribute	Data Type	Description
Product_ID	String	Unique identifier for each product (Primary Key).
Product_Name	String	Name of the product.
Category	String	Product category.
Sub_Category	String	Product sub-category.
Price	Decimal	Selling price of the product.
Supplier	String	Supplier providing the product.

Orders Dataset

Attribute	Data Type	Description
Order_ID	String	Unique identifier for each order (Primary Key).
Customer_ID	String	Customer placing the order (Foreign Key).
Product_ID	String	Product purchased (Foreign Key).
Order_Date	Date	Date when the order was placed.
Quantity	Integer	Number of units purchased.
Payment_Method	String	Payment mode used for the transaction.
Order_Status	String	Current status of the order.

ETL Pipeline Explanation

The project follows the **ETL (Extract → Transform → Load)** process to generate, clean, validate, load, and analyze customer sales data.

1. Extract Phase

File Name	Purpose
generate_customers.py	Generates mock customer dataset.
generate_products.py	Generates mock product dataset.

File Name	Purpose
generate_orders.py	Generates mock order dataset.
data_generator.py	Creates all raw CSV datasets.
data_injection.py	Injects duplicates, missing values, and invalid records into raw data.

2. Transform Phase

File Name	Purpose
customer_cleaning.py	Cleans and validates customer data.
product_cleaning.py	Cleans and validates product data.
order_cleaning.py	Cleans and validates order data.
sales_transaction.py	Creates the Sales Transaction dataset by merging cleaned tables and calculating revenue.

3. Load Phase

File Name	Purpose
database_loader.py	Creates MySQL tables and loads cleaned data into the database.

4. Reporting & Analysis

File Name	Purpose
data_quality_report.py	Generates the Data Quality Report.
business_analysis.sql	Stores business analysis SQL queries.
sql_analysis.py	Executes SQL queries and displays business insights.

5. Pipeline Controller

File Name	Purpose
AppRun.py	Executes the complete ETL pipeline from start to finish.

Data Quality Handling

- **Missing Values:** Detects and handles null or empty values in mandatory fields.
- **Duplicate Records:** Identifies and removes duplicate customer, product, and order records.
- **Data Type Validation:** Ensures fields contain the correct data types such as Date, Integer, Decimal, and String.
- **Email Validation:** Verifies that customer email addresses follow a valid email format.
- **Phone Number Validation:** Ensures phone numbers are valid and properly formatted.
- **Price Validation:** Removes products with invalid or negative prices.
- **Quantity Validation:** Ensures the ordered quantity is greater than zero.
- **Date Validation:** Validates registration and order dates and converts them into a standard date format.
- **Foreign Key Validation:** Ensures every Customer_ID and Product_ID in the Orders dataset exists in the respective Customers and Products datasets.

- **Payment Method Validation:** Accepts only predefined payment methods and rejects invalid values.
- **Order Status Validation:** Validates order status values such as **Completed**, **Cancelled**, and **Pending**.
- **Revenue Calculation:** Calculates revenue only for orders with the **Completed** status.
- **Rejected Records:** Stores invalid records separately for auditing and review.
- **Data Quality Report:** Generates a report summarizing the total, valid, rejected, and cleaned records for each dataset.

Data Transformation and Business Analysis

- **Data Cleaning:** Cleans customer, product, and order datasets by removing invalid, duplicate, and inconsistent records.
- **Data Integration:** Merges cleaned Customers, Products, and Orders datasets to create the final **Sales_Transaction** dataset.
- **Derived Columns:** Generates additional fields such as **Total_Amount**, **Order_Year**, **Order_Month**, and **Year_Month** for reporting and analysis.
- **Revenue Calculation:** Computes total revenue for each completed order using $\text{Quantity} \times \text{Price}$.
- **Database Loading:** Loads the cleaned datasets and **Sales_Transaction** table into MySQL for analysis.
- **Customer Analysis:** Analyzes customer registration, spending patterns, order frequency, and customer distribution by city.
- **Product Analysis:** Identifies top-selling products, highest revenue-generating products, and category-wise sales performance.
- **Sales Analysis:** Evaluates total revenue, average order value, monthly revenue trends, payment methods, and order status distribution.
- **Advanced Business Analysis:** Uses SQL queries with joins, aggregate functions, subqueries, Common Table Expressions (CTEs), and window functions to generate business insights.
- **Business Reports:** Produces meaningful insights to support data-driven decision-making based on customer behavior, product performance, and sales trends.

Logging and Error Handling

- **Centralized Logging:** Uses Python's logging module to record the progress of every ETL pipeline step.
- **Pipeline Monitoring:** Logs the start and completion of each stage, including data generation, cleaning, loading, reporting, and analysis.
- **Error Logging:** Captures exceptions with detailed error messages to simplify debugging and issue tracking.
- **Exception Handling:** Uses try-except-finally blocks to handle runtime errors gracefully and prevent unexpected pipeline failures.
- **Database Transaction Management:** Performs transaction rollback if any database operation fails, ensuring data consistency.
- **Resource Management:** Properly closes database connections and cursors after execution to prevent resource leaks.
- **Log File Generation:** Stores execution logs in the `logs/etl_pipeline.log` file for future reference and auditing.
- **Console Logging:** Displays real-time execution status and error messages in the terminal during pipeline execution.
- **Execution Tracking:** Records timestamps, log levels (INFO, ERROR), and module names for every logged event.
- **Reliable Pipeline Execution:** Ensures the ETL process is traceable, maintainable, and easier to troubleshoot through comprehensive logging and error handling.